

Session 5 speaking script: Concurrency, Coroutines, Modules, and Adoption

How to use this script

- **Bold lines** are the must-say sentences. If you say nothing else on a slide, say those.
- Plain paragraphs are the talk track, written to be read aloud. Short sentences on purpose.
- (pause) and (beat) are deliberate stops. A pause is two full seconds of silence; a beat is one. You are a fast speaker: the pauses are the point, not decoration.
- >> DO: lines are actions (terminal, editor, Compiler Explorer). >> ASK: lines are questions to the room, with the answer you expect and what to say next.
- >> IF AHEAD: is 60 to 120 seconds of real extra depth. >> IF BEHIND: is the one sentence to say instead of the talk track.
- Last session: the close wraps up the whole course, not just today. Protect the last ten minutes.

Pace plan for a fast speaker

Checkpoint	Target clock
Slide 6, What C++11 gave (Concurrency segment starts)	0:10
Slide 20, ThreadSanitizer as a test	0:35
Slide 22, What a coroutine is (Coroutines segment starts)	0:40
Slide 27, Awaitables and <code>co_await</code>	0:52
Slide 34, What modules fix (Modules segment starts)	1:05
Slide 43, Deprecated and removed (Deprecations starts)	1:25
Slide 46, Three tiers (Roadmap workshop starts)	1:30
Slide 49, The worksheet (attendees start writing)	1:35
Slide 51, What is coming in C++26 (Close starts)	1:50
Slide 53 finished	2:00

If you hit a checkpoint more than 3 minutes early, use the IF AHEAD material in the next segment rather than speeding on.

Slow down deliberately on these four:

- **Slide 11, cooperative cancellation.** Three types, a callback that runs on somebody else's thread, and forwarding across two layers. People nod here and then get the exercise's `stop_callback` wrong.
- **Slide 25, generator mechanics.** The `take(2)` lookahead and "destroyed while suspended, never finishes" are the two coroutine facts that cause real bugs. Both need a pause before the answer.
- **Slide 26, what the compiler generates.** The rewrite of the function body is the first time most of the room sees the promise type. Walk it line by line.
- **Slide 27, awaitables.** The demo's punchline (the second half of the function runs on a different thread) is the whole async model. Let the two thread ids sit on screen.

Before class checklist

- Build, from the repo root:
 - `cmake -S . -B build -G Ninja && cmake --build build && ctest --test-dir build -R s05 --output-on-failure`
 - `cmake -S . -B build-tsan -DCOURSE_SANITIZE=thread && cmake --build build-tsan && ctest --test-dir build-tsan -R s05 --output-on-failure` (slow; do it before class, confirm the solution is clean)
 - `cmake -S . -B build-mod -G Ninja -DCOURSE_MODULES=ON` (configure only; build it live on slide 39)
 - Optional for slide 30: `cmake -S . -B build-rel -G Ninja -DCMAKE_BUILD_TYPE=Release && cmake --build build-rel --target demo_s05_generator_perf`
- Check TSan actually runs on this machine: build the race once by hand (command on slide 20) and see the report. On newer Ubuntu kernels TSan can die at startup with `FATAL: ThreadSanitizer: unexpected memory mapping`; the fix is `sudo sysctl vm.mmap_rnd_bits=28`. Find out now, not live.
- Make sure `build/compile_commands.json` exists (configure with `-DCMAKE_EXPORT_COMPILE_COMMANDS=ON` if not). Run the two clang-tidy counts for slide 44 ahead of time and write both numbers on a sticky note as a backup.
- Editor tabs on the right, in this order: `demos/s05/false_sharing.cpp`, `stop_token.cpp`, `semaphore_queue.cpp`, `latch_barrier.cpp`, `atomic_wait.cpp`, `tsan_race.cpp`, `generator_basics.cpp`, `coroutine_machinery.cpp`, `awaitable.cpp`, `state_machine.cpp`, `generator_perf.cpp`, `modules/crc.cppm`, `modules/main.cpp`, `deprecated.cpp`, then `handouts/adoption-roadmap-template.md`.
- Compiler Explorer tabs (the demo headers still say `<add short link>`, so paste the code): (1) `generator_basics.cpp` with `x86-64 GCC 14 -std=c++23` next to `Clang 18 -std=c++23 -stdlib=libc++`, so one prints Fibonacci and the other prints "not available"; (2) `deprecated.cpp` with `-DSHOW_ERRORS -std=c++23` on `GCC 14` and on `Clang 18` with `libc++`, as the backup for slide 44.
- Have the roadmap template ready to paste into the Teams chat, and the path `handouts/adoption-roadmap-template.md` typed in a scratch buffer.
- Read your own notes for slides 15, 17, and 43 below: each has a small correction to the deck you should say out loud.

The script

1. The Evolution of C++, Session 5 • target 0:00, ~1 min

Welcome back. Last session. (beat)

Today is different from the first four. **Everything before today changed how a line of code is written; today changes what a program is.** Concurrency changes who owns a thread and how it stops. Coroutines change control flow. Modules change the build.

And the last forty minutes are about you, not the language: what you actually do on Monday with your own code. (pause)

Same deal as always: the exercise program grows two threads and a coroutine today, and it still prints the Session 1 report, byte for byte.

Here's the plan.

2. Agenda • target 0:01, ~1 min

Seven pieces. Ten minutes of recap and framing. Thirty on concurrency, which is the part you can adopt the soonest. Twenty-five on coroutines. Twenty on modules. Five on what got deprecated and removed while all this was being added.

Then twenty minutes of workshop. **The roadmap worksheet is the only exercise in this course without a solution, because the answer is your codebase.** (pause)

Ten minutes at the end for C++26 and to close out the course.

The exercise README for today has in-class tasks on the bounded queue and the `jthread` pipeline. We won't have a separate block for them; I'll point at them as we go, and they're the first thing to do after class.

First, where Session 4 left us.

IF BEHIND: "Concurrency, coroutines, modules, deprecations, then your roadmap. Let's go."

3. Session 4 recap • target 0:02, ~3 min

The Session 4 solution: projections everywhere, `top_n_by_value` as a filter piped into `to<vector>`, `split` as a pipeline, `chunk_by` in the report, and `records()` returning an `input_range`.

Three places people got stuck. Each one is worth a sentence.

First, `views::split` on an empty string yields zero pieces. The old hand-written loop yielded one empty piece. Neither is wrong. **The tests decided the contract, which is exactly why we wrote the tests first.** (pause)

Second, a `const filter_view` isn't iterable. `begin()` on `filter_view` is non-const because it caches the first match so `begin()` stays amortized constant time. That's by design, on both libraries. The fix is to take ranges by forwarding reference, not by `const&`.

Third, `views::enumerate` isn't in libc++ 18. The portable spelling is `zip of iota` and your `range`.

Two of those three are the lesson for today too. **The standard library is ahead of at least one of your compilers, and a feature-test macro is how you write code that builds on both.** (beat) The support matrix handout is the reference. Today adds a row for nearly everything we show.

ASK: "Who finished the Session 4 take-home tasks?" Expect a few hands. Say: "Good. If you didn't, the Session 5 starter is the Session 4 solution, so you start from a clean place either way."

That's the past. Here's why today's three topics come last.

IF AHEAD: The `const filter_view` issue shows up in code review as "I made the parameter `const auto&` and now it doesn't compile." The review heuristic: a function template that takes a range it only iterates should take `R&&` and constrain it with `std::ranges::input_range`. Taking `const R&` silently rejects every caching view: `filter`, `drop_while`, and `split` on some implementations. If you see `const auto&` on a range parameter in a generic function, ask whether it ever gets a view.

4. Why these three are last • target 0:05, ~3 min

Look at the right-hand column of the table, "cost of adopting." That's the column I want you to read.

Concurrency: **a design review, not a find-and-replace.** You can't sed `std::thread` into `std::jthread` and walk away; the destructor does something different now, and you need to know what.

Coroutines: a new mental model, a function that can pause, and a library to hold it, because the language alone gives you machinery, not types.

Modules: your build system, your CI, and your toolchain floor. In a defense program with a certified toolchain, that floor moves slowly, and that's a real constraint, not an excuse. (pause)

So set expectations. Nobody leaves today a concurrency expert. The goal is narrower: **know what the standard provides now, so the next design doesn't reinvent a semaphore or a stop flag.** Concurrency, adopt now. Coroutines and modules, understand now and adopt with care.

And the telemetry program still has the same contract: a producer thread, a consumer thread, a coroutine, same report. That's harder than it sounds with threads. The moment you split work across threads, ordering becomes something you have to guarantee rather than something you

get for free. The exercise guarantees it with one producer and a FIFO queue, and the diff test proves it.

ASK: "Of the three, which one could your current program actually turn on this year?" Expect "concurrency" or "none, we're on C++17." Say: "Both answers are honest. The roadmap at the end is built for exactly that."

One slide on how we got here.

IF BEHIND: "Concurrency is a design review, coroutines are a mental model, modules are a build change. That's why they're last."

5. The course so far, in one program • target 0:08, ~2 min

Thirty seconds of history, because it's the argument for the whole course.

Session 1: raw new, typedef, iterator loops, sentinel returns, gone. In came `unique_ptr`, `auto`, range-for, optional, `[[nodiscard]]`, the spaceship.

Session 2: `bool` plus an out-parameter became expected. `const string&` became `string_view`. `printf` became `format` and `print`.

Session 3: a runtime CRC table became a `constexpr` table, SFINAE became concepts.

Session 4: iterator pairs and comparator lambdas became projections and pipelines.

Session 5, today: one thread becomes a producer and consumer on `jthread` with semaphores, cancellation by `stop_token`, the parser as a generator, and `ThreadSanitizer` as a test.
(pause)

Same input, same report, every session: report_identical has passed five times.

That's the point. Modernization is incremental and behavior-preserving, or it's a rewrite. And rewrites of long-lived code are how programs slip.

Now the concurrency segment, starting with what C++11 left out.

IF BEHIND: "Five sessions, five diffs, one unchanged report. That's what modernization is supposed to look like."

6. What C++11 gave, and what it lacked • C++11 • target 0:10, ~2 min

C++11 gave us a memory model. That's the big one, and it's easy to forget it: before 2011, threads weren't in the language at all; you were relying on POSIX and your compiler's goodwill. On top of the memory model: `thread`, `mutex`, `lock_guard`, `unique_lock`, `condition_variable`, `atomics`, `future` and `promise`.

Now read the right column. A `std::thread` that's still joinable when it's destroyed calls `std::terminate`. **Forget one `join()`, or throw an exception before it, and the process dies.** (pause)

No cancellation, so every codebase invented an `atomic<bool> stop`. No semaphores, no latch, no barrier: you hand-wrote them on a condition variable. No reader/writer lock. No clean way to lock two mutexes. No way to wait on an atomic without spinning.

ASK: "Hands up if you've written a semaphore on top of a `condition_variable`." Expect most hands. Say: "Everyone has. That's the theme of this segment: the idioms you wrote by hand are now types with specified semantics, and ThreadSanitizer understands them."

C++14 through C++23 fill that right-hand column. Executors and a standard thread pool are still C++26 work; we'll get there.

Why does this matter more than it sounds? Because the hand-written versions don't just cost lines. Each one is a little bit different, written by a different person in a different year, and each one has its own bug surface. A reviewer looking at a homegrown semaphore has to re-derive whether it's correct every time. **A reviewer looking at `std::counting_semaphore` only has to check that it's used correctly.** (pause) In a codebase that lives twenty years and changes hands three times, that difference is most of the value.

First fill: the reader/writer lock.

IF AHEAD: The hand-written semaphore is where a classic bug lives: `notify_one` called without the predicate check, or a `wait` without a predicate, which loses wake-ups or wakes spuriously and proceeds. In review, any `cv.wait(lock)` without a predicate argument is a comment. With the standard semaphore you can't write that bug at all, because there's no predicate to forget.

7. Reader/writer locks: `shared_timed_mutex`, `shared_mutex` • C++14, C++17 • target 0:12, ~2 min

The problem in red: a configuration table read on every record and updated once an hour, and every reader serialized behind one mutex. Readers don't conflict with each other, but a plain mutex doesn't know that.

Look at the right side, `demos/s05/shared_mutex.cpp`. The `limit` function on line 3 of the snippet takes a `std::shared_lock`. Many readers can hold it at once. The `set` function takes a `std::unique_lock` on the same mutex. One writer, and no readers while it holds it.

Note the `mutable` on the mutex. `limit` is `const`, and locking modifies the mutex, so the mutex is `mutable`. That's the one place `mutable` is correct and expected.

C++14 shipped `shared_timed_mutex`. C++17 added `shared_mutex`, which drops the `timed try_lock_for` operations and is cheaper for it. Use `shared_mutex` unless you need the timeouts.

The exercise program has the same shape: a table of per-sensor limits consulted on every record. If the producer and consumer both looked it up, a plain mutex would make them take turns for no reason.

Now the warning. (beat) **A shared mutex is more expensive to acquire than a plain one, so it only pays for read-mostly data.** If your critical section is three instructions, a plain mutex is probably faster. Measure before converting. (pause)

That's a good one for review: "why is this a `shared_mutex`?" deserves an answer with a number in it.

Next, locking more than one mutex.

IF AHEAD: Writer starvation is implementation-defined. The standard doesn't promise a waiting writer gets in while readers keep arriving. glibc's `pthread_rwlock` defaults to preferring readers, and libstdc++ builds `shared_mutex` on it, so a steady stream of readers can starve the hourly writer. If the writer must get through, measure under load, or use the RCU pattern on slide 16, which has no writer starvation because the writer never waits for readers.

IF BEHIND: "`shared_lock` for readers, `unique_lock` for the writer, and only for read-mostly data. Measure first."

8. `std::scoped_lock` • C++17 • target 0:14, ~1 min

Left side, C++11: to lock two mutexes without deadlock, you called `std::lock` on both, then built two `lock_guard`s with `adopt_lock`. Three lines, and the `adopt_lock` tag is the kind of thing people get wrong.

Right side, C++17: one line. `std::scoped_lock lock(from.m, to.m);` Same deadlock-avoidance algorithm underneath, for any number of mutexes. CTAD deduces the mutex types.

The one-mutex case matters more than the two-mutex case: use `scoped_lock` everywhere and there's one spelling in the codebase. (pause)

One trap. `std::scoped_lock lock;` with no mutex compiles. It deduces `scoped_lock<>` and locks nothing. GCC with `-Wall` flags it as an unused variable, which under the repo's `-Werror` breaks the build. Its sibling, `std::scoped_lock{m};` with braces and no name, creates a temporary that unlocks at the semicolon; clang-tidy's bugprone-unused-raii catches that one.

Now a performance bug with no lock in sight.

IF BEHIND: "One line, deadlock-free for any number of mutexes. Make it the default."

9. False sharing and `hardware_destructive_interference_size` • C++17 • target 0:15, ~2 min

Problem statement: two counters, two threads, no shared data, and it's several times slower than it should be.

Look at Packed on the right. Two `atomic<long>`s side by side. Sixteen bytes. They fit in one 64-byte cache line. Thread A writes a, thread B writes b. They never touch each other's variable. (beat)

ASK: "No shared variables. Why is it slow?" Wait for "cache line" or "cache coherence." Say: "Right. **The cache line is the unit of coherence, not the variable.** Each core's write invalidates the other core's copy of the whole line, and the line pings-pongs."

Padded puts each atomic on its own line with `alignas(kLine)`. `kLine` is `hardware_destructive_interference_size` where the library defines it, 64 on x86. `libc++ 18` doesn't define it, so the demo falls back to 64.

DO: In the terminal: `./build/demos/s05/demo_s05_false_sharing`. Expect one line like `packed 695ms padded 171ms (sizeof 16 vs 128)`. Point at the two times, then at the sizes: "we paid 112 bytes to get that back." The deck's numbers are from GCC 14 on x86-64; yours will differ, the ratio is what matters. Recovery: if the ratio is small, the two threads probably shared a core or the VM has one vCPU. Say "on a real multicore box this is about four to one," show the deck's numbers, and move on.

This is a Session 1 callback, by the way: `alignas` is C++11. The new part is the named constant.

Next: a thread that cleans up after itself.

IF AHEAD: Two details. First, some ARM cores have 128-byte lines, and Intel's adjacent-line prefetcher pulls pairs of 64-byte lines, which is why libraries like folly pad to 128. Second, GCC can warn with `-Winterference-size` when the constant is used in a header, because its value can change with `-mtune` and that makes it an ABI hazard. For anything in a public struct, define your own `constexpr` constant and document it. In a per-thread stats array this is the fix; in a struct that crosses a library boundary, it's an ABI decision.

IF BEHIND: Skip the demo. "Two atomics on one cache line ping-pong between cores; `alignas` the interference size apart. The deck's numbers show about four to one."

10. `std::jthread` • C++20 • target 0:17, ~2 min

Left side, C++11, `demos/s05/jthread.cpp`. A worker that loops until an `atomic<bool>` flag goes true. Look at the last line of `run11`: `t.join()`, with the comment "mandatory; an exception before it terminates." If anything between the constructor and that join throws, the `std::thread` destructor runs on a joinable thread and calls `std::terminate`. (pause) Not an exception, not a log line. Terminate.

Right side, C++20. `std::jthread`, the joining thread. The lambda takes a `std::stop_token` as its first parameter. You don't pass it; **`jthread` hands the callable its own token automatically if the callable accepts one.** The loop checks `st.stop_requested()`.

And look where the function ends: the closing brace with the comment "`~jthread`: `request_stop()`, then `join()`." In that order. **That turns "forgot to join" from a crash into correct behavior, and it gives every thread a cancellation channel for free.** (pause)

From the exercise: `std::jthread producer([&](std::stop_token producer_stop) { ... })`; Same shape.

The cost is a small shared stop-state allocation per thread. For anything that isn't creating threads in a hot loop, that's nothing.

One honest caveat. `jthread`'s destructor joins. A thread that never checks its token and is blocked forever in a `read()` will now hang your destructor instead of crashing it. That's usually better, but it's a behavior change, so look at each thread's exit path when you convert.

Now what that token actually is.

IF AHEAD: Member order matters with `jthread`. Members are destroyed in reverse declaration order. If a class holds a `jthread` whose lambda uses other members, declare the `jthread` last, so it's destroyed, and therefore joined, first, while the members it uses are still alive. Declare it first and the thread runs for a moment against destroyed members. That's a review comment you'll write more than once.

IF BEHIND: "`jthread` requests a stop and joins in its destructor; the token arrives as the first parameter. Replace `std::thread` with it, checking each thread's exit path."

11. Cooperative cancellation: `stop_token`, `stop_source`, `stop_callback` • C++20 • target 0:19, ~3 min

Slow down here. Three types. **`stop_source` requests a stop. `stop_token` observes one. `stop_callback` runs a function when a stop is requested.** (pause)

Read `demos/s05/stop_token.cpp` from the bottom. `main` makes a `stop_source` on line 1 of `main`. A `jthread` runs pipeline with `source.get_token()`. A token is a cheap view of the source; copy it as much as you like. Twenty milliseconds later, `source.request_stop()`. One call.

Now go up into `pipeline`. It receives the outer token. It starts a producer `jthread` with its own token, from its own `jthread`. Two separate stop states. How does the outer stop reach the producer? (beat)

The `stop_callback` named `forward`, on the line after the producer. It registers a lambda against the outer token: when the outer stop is requested, call `producer.request_stop()`.

Here's the subtle part. **The callback runs on whichever thread calls `request_stop()`.** In this demo, that's the main thread, reaching into `pipeline`'s producer. And if the stop was already requested when the callback is constructed, it runs immediately, in the constructor, on the constructing thread. (pause)

Then `pipeline` sees its own outer token stopped, returns, and its locals unwind in reverse order: the callback is deregistered first, then the producer `jthread` joins. One request unwound both threads.

Why not a `bool`? An `atomic<bool>` isn't wrong; it's thread-safe too. The problem is it doesn't compose. You can't forward it across layers without writing this callback mechanism yourself, and `condition_variable_any::wait` takes a `stop_token` directly, so a blocked wait wakes up when the stop arrives. A `bool` can't wake anything.

The exercise uses exactly this. `load_and_compute` takes a token from its caller, and a `stop_callback` requests the producer's stop and closes the queue.

DO: Optional, 20 seconds: `./build/demos/s05/demo_s05_stop_token`. Expect producer stopped after N iterations with N somewhere around 15 to 20. Point out that main never touched the producer. Recovery: none needed; if it hangs, Ctrl+C and say "and that is why the callback matters."

Next: the blocking primitive the exercise is built on.

IF AHEAD: The callback's destructor is careful. If the callback is running on another thread at the moment it's destroyed, the destructor blocks until it finishes. That's why the declaration order in `pipeline` is safe: `forward` is destroyed before `producer`, so the callback can never fire on a dead `j` thread. Swap those two declarations and the callback could call `request_stop()` on a destroyed object. Order of locals is a correctness property in this code, and worth a comment in the source.

12. counting_semaphore and binary_semaphore • C++20 • target 0:22, ~2 min

"Wait until there's a slot" and "wait until there's an item" used to be a condition variable and a predicate. Now they're counts.

This is a hand-typed excerpt from the exercise solution's `queue.h`. Two semaphores. `slots_` starts at `Capacity`: the free space. The producer acquires one per push. `items_` starts at zero: the queued elements. The consumer acquires one per pop.

Look at `push`. `slots_.acquire()` blocks while the queue is full. Then a short `lock_guard` around the `std::queue` push. Then `items_.release()`.

Look at `pop`. It doesn't just acquire. It loops on `try_acquire_for(10ms)` and checks the stop token in between. That's how a stop is noticed while the consumer is blocked. A plain `acquire` would sleep through it.

ASK: "If the semaphores do all the blocking, why is there still a mutex?" Pause for it. Expected: "the queue itself isn't thread-safe." Say: "Exactly. **The semaphores count; the mutex guards the std::queue.** With one producer and one consumer, a push and a pop can still touch the same deque at the same time."

Think about what the two counts mean together. `slots_` plus `items_` always adds up to `Capacity`, give or take the operations in flight. The producer turns a slot into an item; the consumer turns an item back into a slot. That invariant is the whole design, and you can say it in one sentence in a code review. With the condition-variable version, the invariant is spread across two predicates and two notify calls.

API: `acquire`, `release` with an optional count, `try_acquire`, `try_acquire_for`. The count type is `std::ptrdiff_t`, signed. The exercise builds with `-Wsign-conversion`, so a `size_t` capacity gets flagged. `binary_semaphore` is just `counting_semaphore<1>`.

Side by side with the C++11 version next.

IF AHEAD: The template argument is `LeastMaxValue`, a minimum the implementation must support, and releasing past `max()` is undefined behavior. That's why the exercise sizes `items_` as `Capacity + 1`: `close()` releases one extra count to wake a blocked consumer, and if the queue is already full, that's one more than `Capacity`. Get the size wrong and it works in testing and is UB the one time `close()` races a full queue.

13. The bounded queue, before and after • C++20 • target 0:24, ~2 min

`demos/s05/semaphore_queue.cpp`. Left is C++11 style: one mutex, two condition variables, `not_full_` and `not_empty_`, and a predicate lambda in each wait. Right is C++20: the mutex and two semaphores.

Walk push on both. Left: take a `unique_lock`, wait on `not_full_` with the predicate `q_.size() < N`, push, notify one on `not_empty_`. Right: `slots_.acquire()`, lock, push, unlock, `items_.release()`.

Same structure. **Two fewer things to get wrong: no predicate to forget, and no notify on the wrong condition variable.** (pause) In review, the left side needs a careful reader. The right side reads like the comment that describes it.

One honesty note about the left side: it's labeled C++11 but it uses `std::unique_lock lock(m_)` without the template argument, which is CTAD, a C++17 feature. Real C++11 code writes `std::unique_lock<std::mutex>`.

DO: Optional, 10 seconds: `./build/demos/s05/demo_s05_semaphore_queue`. Expect 45, the sum of 0 to 9 through a queue of four slots. Recovery: not needed.

The exercise's version adds `close()`: set a flag, then release one extra `items_count` so a blocked pop wakes, sees the flag, and returns `nullopt` once drained. That's task 1 in the README.

Next: two primitives for groups of threads.

IF BEHIND: "Same structure, the semaphores replace both condition variables and both predicates."

14. `std::latch` and `std::barrier` • C++20 • target 0:26, ~2 min

Two shapes of group synchronization. "Wait for all N workers to start." And "nobody begins step 2 until everyone finishes step 1."

Top snippet, the latch. A one-shot countdown, initialized to four. Each worker calls `count_down()`. The main thread calls `ready.wait()` and blocks until it hits zero. **A latch can't be reset: it's the fan-in half of fan-out, fan-in.**

Bottom snippet, the barrier. Reusable. Each worker runs three steps and calls `arrive_and_wait()` after each one. When the last thread arrives, the completion function runs exactly once for that phase, before anyone is released. It must be `noexcept`. So the output says phase 0, 1, 2 complete.

DO: `./build/demos/s05/demo_s05_latch_barrier`. Expect four "worker N ready" lines in some order, "all workers ready," then "phase 0 complete," "phase 1 complete," "phase 2 complete." Point out: "all workers ready" can print before the last "worker N ready" line. (pause) Ask why. Answer: each worker counts down *before* it prints, so main can wake first. Recovery: if the order happens to be tidy, say "run it a few times; the order isn't guaranteed, which is the lesson."

Look at the declaration order in the barrier part: phase, then sync, then steppers. The `pthread_vector` is destroyed first, so the threads join while the barrier still exists. Reverse those and you destroy a barrier threads are waiting on.

Next: waiting without any of these types.

IF AHEAD: `arrive_and_drop()` lets a thread leave the group for all future phases, which is how a worker that runs out of data stops participating without deadlocking the others. And the completion function runs on one of the participating threads, unspecified which, so it shouldn't assume it's on main. That's also why `phase++` in the demo is safe: only one completion runs at a time, and nobody else touches phase.

IF BEHIND: Skip the demo. "Latch: one-shot countdown. Barrier: reusable, with a once-per-phase completion. Both replace a mutex, a condition variable, and a counter."

15. `atomic::wait / notify` and `atomic_ref` • C++20 • target 0:28, ~2 min

Waiting for an atomic to change used to mean spinning, or a mutex you only held so you could sleep on a condition variable.

Top snippet. `state.wait(0)` means "block until the value is no longer zero." Then the main thread stores 1 and calls `notify_one`. **The atomic is the wait point.** On Linux, `libstdc++` implements it with a futex, after a short spin, so it's a real sleep. And it's what `libstdc++`'s semaphore, latch, and barrier are built on.

Note there's no lost wake-up here. If the store happens before the waiter even calls `wait(0)`, the value isn't zero, so `wait` returns immediately.

Bottom snippet, `atomic_ref`. A Legacy struct with a plain `int`: think of a struct from a C library or a hardware interface header you can't change. `std::atomic_ref<int> ref(obj.counter)` does atomic operations on that plain `int`, in place. Two threads, a thousand `fetch_adds` each, and the result is 2000. The object never changed type.

The rule in the comment matters: **while any `atomic_ref` to an object exists, touch it only through `atomic_refs`.** (pause) And look at the print on the last line. It reads `obj.counter` directly while `ref` is still in scope. After the joins there's no actual race, but by the letter of the standard that read should be `ref.load()`, or `ref` should live in an inner scope. That's precisely the kind of comment a good reviewer leaves.

ASK: "What happens if the object isn't aligned the way an atomic needs?" Expect a guess. Say: "It's a precondition: the object must meet

`atomic_ref<T>::required_alignment`. For an `int` that's natural alignment. For a packed struct off the wire, check it."

libc++ 18 doesn't have `atomic_ref`; the demo gates on `__cpp_lib_atomic_ref`.

Next: the same idea for a whole configuration object.

IF AHEAD: `wait(old)` compares values, so it's subject to ABA: if the value changes from 0 to 1 and back to 0 before the waiter looks, the waiter may keep sleeping. For a state flag that only moves forward, that's fine. For a counter that wraps, use a generation number. And `wait` on types that aren't four bytes goes through a hashed table of waiters in libstdc++, so it works but can wake unrelated waiters and costs more.

IF BEHIND: "wait and notify make an atomic a sleep point; `atomic_ref` makes a plain int atomic in place. Touch it only through the ref while the ref exists."

16. `std::atomic<std::shared_ptr<T>>` • C++20 • target 0:30, ~1.5 min

The configuration table again, from slide 7. Readers on every record, a writer once an hour, and this time no lock on the read path at all.

`g_config` is an `atomic<shared_ptr<const Config>>`. The reader on line 5 does one atomic load and gets its own `shared_ptr` copy. That copy keeps the `Config` alive for as long as the reader needs it. The writer, `reload`, builds a completely new `Config` and stores it. **In-flight readers keep the old one; it dies when the last reader drops it.** That's read-copy-update. (pause)

Two honesty points. It's not lock-free on most implementations; libstdc++ uses a small internal lock. But the read path never waits on the writer's work, only on a pointer swap. And the global starts out null, so something has to call `reload` before the first limit; the demo does that on its first line.

The C++11 free functions, `std::atomic_load` of a `shared_ptr*` and friends, are deprecated in C++20. They made a non-atomic `shared_ptr` look atomic if you remembered to use them everywhere. Nobody remembers everywhere.

libstdc++ 12 and later have it; libc++ 18 doesn't.

Next: logging from all these threads.

IF BEHIND: Fold into the previous slide: "Hot-swappable config is `atomic<shared_ptr<const T>>`: readers copy, the writer swaps."

17. `std::osyncstream` • C++20 • target 0:31, ~1 min

Four threads write to `cout` with three `<<` each. The output comes out interleaved.

Read the slide's red line: the standard only promises that concurrent use of `cout` isn't a data race. It promises nothing about the line; characters from different threads can interleave. **The line isn't atomic.**

The fix: wrap `std::cout` in an `osyncstream` for the statement. It buffers everything written to it and transfers the whole buffer to `cout` in one go when it's destroyed, here at the end of the full expression, or when you call `emit()`. No lock in your code.

Pair it with Session 2's format, or call `std::print` with the `osyncstream` as the stream.

DO: Optional, 10 seconds: `./build/demos/s05/demo_s05_osyncstream`. Four clean lines in some order. Recovery: none needed.

On `libc++ 18` it's behind `-fexperimental-library`, which the repo's CMake adds.

Next: a callable type every task queue needed.

18. `std::move_only_function` and task queues • C++23 • target 0:32, ~1.5 min

A task queue of `std::function` can't hold a task that owns a `unique_ptr`. `std::function` requires a copyable callable, and a lambda that captured a `unique_ptr` by move isn't copyable.

Line 1 of the snippet: a queue of `move_only_function<void()>`. Line 3 pushes a lambda that owns the payload. Line 5 pops by moving and calls.

The old workaround was to capture a `shared_ptr` instead, which made the task copyable by lying about ownership. **It's Session 2's lesson, ownership in the type, applied to callables.** (pause)

It also carries qualifiers in the signature: `move_only_function<void() const noexcept>` means the call operator is `const` and `noexcept`. `std::function` never could say that.

`libc++ 18` doesn't have it; the demo gates on `__cpp_lib_move_only_function`.

ASK: "What do you get if you call an empty `std::function`?" Expect "bad_function_call." Say: "Right. Call an empty `move_only_function` and it's undefined behavior. No exception. Check before you call if empty is possible."

Now the honest slide: what still isn't there.

IF BEHIND: "A queue of `move_only_function` holds tasks that own `unique_ptr`s; `std::function` can't."

19. What is still missing, and where it is going • target 0:34, ~1.5 min

C++23 has no thread pool, no executor, no async I/O. Every project still picks one.

`std::async`: the future it returns blocks in its destructor. No pool, no priority, no cancellation. **Fine for a one-off; don't build on it.** A common bug is calling `std::async` and ignoring the returned future: the temporary future's destructor waits right there, and your "asynchronous" call is synchronous. (pause)

Thread pools are still a library choice: TBB, Asio, folly, or your own on `jthread` plus the bounded queue from the exercise. If you write your own, it's maybe sixty lines with what we've

seen today: a vector of `jthreads`, the bounded queue holding `move_only_function<void()>`, each worker popping with its own stop token. Shutdown is free, because destroying the vector requests a stop on every worker and joins them all. That's the slide-10 destructor doing real work.

`std::execution`, senders and receivers, P2300, is C++26. That's the composable model, and the one `co_await` will plug into. Parallel algorithms from Session 4 are the only built-in parallelism, and on GCC they need TBB to actually run in parallel. Hazard pointers and RCU are C++26 too.

What to do now: `jthread` for lifetime, `stop_token` for cancellation, a semaphore queue for hand-off, and a pool behind one interface you can swap later.

Now the tool that tells you whether any of this is right.

IF BEHIND: "No pool or executor until C++26; keep yours behind an interface so `std::execution` can slot in."

20. ThreadSanitizer as a test • demo • target 0:35, ~3.5 min

`demos/s05/tsan_race.cpp`. A plain `int`, written a thousand times by a writer `jthread`. Under `SHOW_ERRORS`, main reads it before the join. That's a data race. Without `SHOW_ERRORS`, main reads it after `writer.join()`. **`join()` is a synchronization point: every write in the thread happens-before the code after the join.** No race. (pause)

The exercise's `lines_read` counter is exactly this shape: written by the producer, read by the consumer. Read it before the join and it's a race. That's task 3.

ASK: "Does TSan need the race to actually collide at runtime to catch it?" Let them guess. Answer: "No. It tracks happens-before. If two accesses aren't ordered, it reports them, even on a run where the timing happened to be harmless. **It's a checker, not a stress test.**"

DO: In the terminal, build the racy version by hand: `g++-14 -std=c++23 -g -fsanitize=thread -DSHOW_ERRORS demos/s05/tsan_race.cpp -o /tmp/tsan_race && /tmp/tsan_race`. Expect WARNING: ThreadSanitizer: data race, then two stacks, a write in the lambda and a read in main, plus the thread creation site. Point at "Previous write ... by thread T1" and "Read ... by main thread." Then `echo $?`: TSan's default exit code on a report is 66. Non-zero, so CTest fails. Recovery: if it dies with unexpected memory mapping, that's the kernel ASLR setting, not your code; say so, and show the report from the pre-built run or skip to the exercise run.

DO: Then the exercise under TSan, already built: `ctest --test-dir build-tsan -R s05 --output-on-failure`. Expect all s05 tests passing, no warnings. Say: "That's the solution, clean on GCC 14 and Clang 18." Recovery: if tests fail, the build-tsan tree is stale; don't debug live, say "I'll post the clean run in the chat."

TSan understands every primitive we showed today: joins, semaphores, latches, atomics. That's why "every concurrent test runs under TSan in CI" can be a policy, not a hope. The cost is a

slowdown of roughly 5 to 15 times and 5 to 10 times the memory, so it's a separate CI job, not the default build.

Takeaway next.

IF AHEAD: TSan and ASan can't be combined in one binary; they each want the address space. That's why the course runs them as separate jobs. And TSan needs a 64-bit host: Linux, macOS, or FreeBSD. It won't run on your RTOS target. The pattern for embedded teams is to keep the threading logic portable enough to build in host unit tests and run TSan there, which is another argument for standard primitives over RTOS-specific ones in the logic layer.

IF BEHIND: Skip the exercise run. Show only the racy run, point at the warning and the exit code, and move on.

21. Concurrency takeaway • target 0:39, ~1 min

jthread plus stop_token for lifetime and cancellation. Semaphores for hand-off. latch and barrier for phases. scoped_lock for every lock. ostream for logging.

The hand-written versions of all of these are in your codebase right now. (beat) Each replacement is a design review, and ThreadSanitizer is how the review ends. (pause)

For tonight: exercise tasks 1 through 3 are the bounded queue, the jthread pipeline, and the TSan run. Start there.

The queue hands records between threads. The next segment hands them between a function and its caller, with no thread at all.

22. What a coroutine is • C++20 • target 0:40, ~2.5 min

The definition in red: a function that can suspend in the middle, return to its caller, and later resume where it left off.

Four facts.

One. **Any function whose body contains co_await, co_yield, or co_return is a coroutine. Nothing in the signature says so.** You find out by reading the body. In review, that's worth knowing.

Two. Its locals live in a coroutine frame, usually heap-allocated, that survives each suspension. That's why the locals are still there when you come back.

Three. (beat) Suspending is not blocking, and it's not a thread. The caller gets control back, on the same thread.

ASK: "If I call a coroutine from main, which thread runs its body?" Pause. Expect "main" and maybe "a new thread." Say: "Main, until something else resumes it. **A coroutine is a control-flow feature, not a concurrency feature.** It runs on whichever thread resumes it. We'll see that on slide 27 with a demo that changes threads in the middle of a function."

Four. C++20 shipped the language machinery, `<coroutine>`, `coroutine_handle`, `suspend_always`, and no library types at all. You wrote a promise type, or you used a library. That's why C++20 coroutines were nearly unusable for most teams. C++23 shipped the first library type: `std::generator`.

The language gives the mechanism. The policy, what happens at a suspend and who resumes, lives in a type someone writes once.

A useful way to hold it in your head: a normal function's locals live on the stack, so they die when the function returns. A coroutine's locals live in the frame, so they survive a "return" that's really a suspend. **The frame is an object, with a lifetime, owned by whoever holds its handle.** (pause) Every coroutine bug we'll talk about today is a lifetime bug about that object: who destroys it, when, and what it was pointing at.

Why do we care, for this program?

IF AHEAD: Things that can't be coroutines: constructors, destructors, `main`, `constexpr` and `constexpr` functions, and functions with C-style varargs. And a coroutine can't use a plain `return`; it must be `co_return`. Also, these are stackless: only the coroutine's own frame is saved. A function it calls can't suspend on its behalf unless that function is itself a coroutine that's awaited. That's the difference from stackful fibers like `Boost.Context`.

23. Why they matter here • C++20 • target 0:42, ~1.5 min

Four rows. A lazy sequence, the parser: without coroutines it's an iterator class with hand-kept state, or an eager vector. With them, `co_yield` in a loop.

A state machine, like a protocol decoder: without, an `enum State` and a `switch`. With, **the state is the program counter.** (pause)

Async I/O: without, callbacks and the state they drag along in captures. With, `co_await` the operation and the code reads top to bottom. That's where most of the industry's coroutine use is today, through `Asio` and `folly`, and it's exactly where the standard library still has nothing until `std::execution`.

Embedded cooperative scheduling: suspend and resume with no stack per task, instead of `setjmp` or an RTOS task.

What they're not: parallelism. Two coroutines on one thread never run at the same time. Pair them with the last segment for that.

We'll see three of these rows today: the parser, the decoder, and a taste of async. Parser first.

IF BEHIND: "Lazy sequences, state machines, async I/O; not parallelism."

24. `std::generator` • C++23 • target 0:44, ~2.5 min

Left: the exercise's Session 4 parser, `load_stream`. Read every line, strip the carriage return, parse, push accepted records into a vector, count rejects in a map, return everything at the end. Eager. Nothing reaches the caller until the whole file is parsed.

Right: the Session 5 solution, records. Same loop. Look at line 6 on the right: instead of `push_back, co_yield std::move(*r)`. One record per yield. **The caller pulls; nothing runs until it does.**

And the return type is `std::generator<Record>`. It's an `input_range`. So the bottom two lines pipe it into `views::take(2)` and `loop`. **Every Session 4 view composes with it.** (pause)

Now notice what got lost. (beat) The rejected-line count. A generator yields one kind of thing. That's a design question, not a bug. You can yield a `variant<Record, ParseError>`, the way the exercise's queue does, or keep both entry points, which is what the solution does.

ASK: "What's the classic lifetime bug with a function like this that takes `std::istream&?`" Expect "the stream dies before the generator is consumed." Say: "Yes. A reference parameter is stored in the frame as a reference. If the generator outlives the argument, it dangles. The Core Guidelines have a rule for it, CP.53: don't pass coroutine parameters by reference unless you control the lifetime."

Look at the size of the change between left and right. The loop body is almost identical. The vector, the map, and the return statement are gone. That's typical: a generator is usually the eager function with the container deleted. **If you can write the eager loop, you can write the generator.** The hard part isn't the syntax; it's the lifetime questions on the next slide.

This is exercise task 5. It's gated on `__cpp_lib_generator`: `libstdc++ 14` has it, `libc++ 18` doesn't.

How lazy is it, exactly?

IF AHEAD: The sharpest version of CP.53 is `string_view`. A generator taking `std::string_view` and called with a temporary `std::string`: the string dies at the end of the full expression that created the generator, and every later resume reads freed memory. ASan catches it; the type system doesn't. For generators, take owning parameters, `std::string` by value, unless the caller provably outlives the generator.

25. Generator mechanics: lazy, and the lookahead • C++23 • target 0:46, ~3 min

Slow down. Two consequences on this slide cause real bugs.

Top snippet, `demos/s05/generator_basics.cpp`. `fields` splits a line on commas and yields each piece as a string. `use` asks for `fields("1,rpm,40,extra") | take(2)`. Prints "1" and "rpm." The comment says the third field is never built. Let's be precise about that in a moment.

Middle snippet: `fibonacci` with `while (true)`. Infinite. That's fine, because it's lazy.

DO: `./build/demos/s05/demo_s05_generator_basics`. Expect 1, rpm, then 55 89 144 233 377 (Fibonacci from index 10, five values), then eager: 3 fields. Recovery: if it prints "not available," you're on the `libc++` build; switch to the GCC tab in Compiler Explorer.

Now the exercise's test. Read `records(in) | views::take(2)` and stop. Then ask the stream which line it's on.

ASK: "After taking two records, what's the next unread line in the file: 3 or 4?" Wait for votes. (pause) Then: "Four."

take(2) resumes the coroutine once more than it uses. When the loop advances past the second element, `take`'s counted iterator increments the underlying iterator first, which resumes the generator, which reads line 3 and yields record 3. Then the count hits zero and the loop stops. It's the same lookahead as Session 4's laziness demo. For `fields`, that means the third string actually was built; it just wasn't used. (pause)

Second consequence. After the loop, the generator is destroyed **while suspended. Its locals are destroyed, and nothing after that `co_yield` ever runs.** (pause)

Think about what that means. If the coroutine holds a lock across a `co_yield`, the lock stays held until the frame is destroyed. If there's a "close the file" or "log the count" after the loop, it never runs. **Cleanup goes in destructors, not after the loop.** That's the same RAII rule as always, but coroutines make the "after the loop" code look safe when it isn't. Core Guidelines CP.52 says it directly: don't hold locks across suspension points.

And one more: the body runs on the consumer's thread, at the consumer's pace. There's no thread anywhere in this.

How does the compiler do this?

IF AHEAD: The `take`-lookahead was argued about in committee: there was a paper to add a counted iterator that doesn't advance past the last element, and it didn't make C++23. If the extra resume matters, a byte stream where you need to leave the stream exactly positioned, don't use `take` on a generator; write the loop with a counter and break before advancing.

26. What the compiler generates • C++20 • target 0:49, ~3 min

Slow, again. This is a minimal hand-written generator, `demos/s05/coroutine_machinery.cpp`, so `std::generator` isn't magic. **Never write this in production.** It has no allocator support, no exception propagation, no nested yields.

Top snippet. The class `Gen<T>` has a nested type called exactly `promise_type`. **The compiler looks for that nested type by name; every customization point lives there.**

Walk it. `value` holds the last yielded value. `get_return_object` builds the `Gen` the caller receives, from a handle to this promise. `initial_suspend` returns `suspend_always`: lazy, do nothing until the first resume. `final_suspend` also suspends, so the frame survives and `done()` can be read. `yield_value` stores the value and suspends: that's what `co_yield v` turns into. `return_void` handles falling off the end. `unhandled_exception` here just terminates.

Now the bottom snippet, the call. `counter(3)`. Here's the rewrite the compiler does, in order. (beat)

Allocate the frame. Construct the promise inside it. Call `get_return_object`: that's what the caller gets back as `g`. Then `co_await initial_suspend()`, which suspends. So when `counter(3)` returns, **not one line of the body has run**. (pause)

`g.next()` resumes. The body runs to `co_yield i`, which is `co_await promise.yield_value(i)`: store 0, suspend. next returns not-done. Print 0. Again for 1 and 2. On the fourth resume, the loop ends, `return_void`, then `co_await final_suspend()`, and now `done()` is true.

The handle half, which isn't on the slide, is in the file: the Gen destructor calls `handle.destroy()`, which frees the frame. That's the "destroyed while suspended" from the last slide, made concrete.

DO: `./build/demos/s05/demo_s05_coroutine_machinery`. Expect 0 1 2 .
Say: "Twenty-odd lines of plumbing to print three numbers. That's the argument for `std::generator`." Recovery: none needed.

Now the other direction: what `co_await` does.

IF AHEAD: One quirk worth knowing. The order matters: `get_return_object` is called before `initial_suspend`, and when its result is converted to the function's return type varies between compilers, eagerly on some, after the first suspension on others. For a generator it doesn't matter. For a coroutine that wants to put an error into its own return value, which is the next-but-two slide, it matters a lot, and it's one reason those types are hard to write portably.

IF BEHIND: Skip the run. "The compiler allocates a frame, builds the promise, returns the object, then suspends before the body. Every hook is a member of `promise_type`."

27. Awaitables and `co_await` • C++20 • target 0:52, ~3 min

`co_await x` asks `x` three questions. **Are you ready? Where do I park? What's my value?** Those are `await_ready`, `await_suspend`, and `await_resume`.

Top snippet, `demos/s05/awaitable.cpp`. `ValueFromThread`. `await_ready` returns true if the value's already there, and then there's no suspension at all. `await_suspend` receives the coroutine's handle and starts a `jthread`. That thread sets the value to 42 and calls `h.resume()`. `await_resume` returns the value.

Bottom snippet, `consumer`. It prints its thread id. Then `co_await ValueFromThread{}`. Then prints the value and its thread id again.

ASK: "Which thread prints the second line?" Pause. Let someone commit. Expect a split.

DO: `./build/demos/s05/demo_s05_awaitable`. Expect two lines: before `await` on `<id A>` and after `await`: 42 on `<id B>`, with A and B different. Point at the two ids and stop talking for a moment. Recovery: if the ids happen to look similar, read them out digit by digit; they differ. If it doesn't build, use the description and move on.

(pause)

The code after `co_await` runs on the producing thread, because that's who called `resume()`. That's the whole model of async coroutines: whoever completes the operation continues the function. Same function, two threads, no lock you can see.

And that's why hand-writing these is dangerous. Lifetimes: the awaiter lives in the frame. Exceptions. Which thread you're on after every `co_await`. The comment on the global producer says it: a real library would own that thread properly.

The Task type in the file uses `suspend_never` at both ends: it runs eagerly and frees its own frame when the body finishes, on whatever thread that happens on. Main waits on a `binary_semaphore` that the coroutine releases at the end.

In production: `std::generator`, a library like `cppcoro`, folly's `coro`,asio's awaitables, `libunifex`, or C++26's `std::execution`.

Now a use that pays off even on one thread.

IF AHEAD: There's a race hiding in this pattern. The new thread can call `h.resume()` before `await_suspend` has even returned on the original thread. The standard allows that: the coroutine is already considered suspended when `await_suspend` is called. The rule is that after starting the work, `await_suspend` must not touch the awaiter or the frame, because the frame may already be gone. Here it only assigns to a global, so it's fine. Also, `await_suspend` can return `bool`, false meaning "don't suspend after all," or another `coroutine_handle` to resume directly, which is symmetric transfer and how libraries avoid stack overflow in long chains.

28. A coroutine as a state machine • C++20 • target 0:55, ~2.5 min

A frame decoder: sync byte 0xAA, a length, the payload, a checksum byte. Fed one byte at a time, which is how a serial or bus driver hands you data.

Left, `SwitchDecoder`. The state is data: an enum, a length, a running sum, and a partial buffer. Look at the `Len` case: it has to clear the payload, zero the sum, and pick the next state.

Every transition is a line you must not forget. A common bug is a missing `sum_ = 0` on one path, which passes every test that starts from a fresh decoder and fails on the second frame. (pause)

Right, `decode()`. Read it like the spec. Wait for 0xAA. Read the length. Loop `len` times, reading bytes and summing. Read the checksum; if it matches, `co_yield` the frame. **The for loop is the Payload state.** The locals reset themselves because they're declared fresh each time around.

The forty lines of plumbing, the `Decoder` type with `next_byte`, are in the file, not on the slide. Written once per project.

DO: `./build/demos/s05/demo_s05_state_machine`. Expect switch: frame of 2 bytes, coroutine: frame of 2 bytes, switch: frame of 0 bytes,

coroutine: frame of 0 bytes, then 2 frames each (the third has a bad checksum). Point at the wire bytes in main: the third frame's checksum is 0x06 against a sum of 0x05. Recovery: none needed.

This is where coroutines pay off in embedded code: protocol decoders, parsers, anything that's "wait for the next input." Single-threaded, small, and testable.

IF AHEAD: The "before" side uses `using enum`, which is C++20, so it isn't strictly C++11; the comment says so. More useful: the coroutine version has one frame allocation for the life of the decoder, at construction, not per byte. If your coding standard forbids heap after initialization, that can still be acceptable, and if it isn't, `promise_type` can define its own `operator new` to allocate from a static pool. That's the hook for no-heap targets.

IF BEHIND: Skip the demo. "The switch keeps the state in data; the coroutine keeps it in the program counter, and reads like the spec."

29. Coroutines and expected • C++23 • target 0:58, ~1 min

The Session 2 preview, closed out. This is not standard C++23; it's a custom promise type.

`build` returns an `Expected`. `co_await parse_header(in)` means: unwrap the value, or return the error from the whole function. **Early return on error, with no macro and no `and_then` chain, because the promise type sees every `co_await`.**

The mechanism: the promise's `await_transform` turns each `expected` into an awaitable whose `await_ready` is `has_value()`; on error, `await_suspend` stores the error in the result and destroys the frame.

`Boost.Outcome` and a few proposals explore this; nothing's standard. C++26's `std::execution` takes a different route: errors are a channel, `set_error`, so `co_await` on a sender propagates them. (pause)

IF BEHIND: Skip the slide. "A custom promise can make `co_await` on an `expected` mean early return; not standard, and `std::execution` does errors differently."

Next: what it costs.

30. Allocation and performance • C++23 • target 0:59, ~2 min

Three ways to parse two million comma-separated integers, `demos/s05/generator_perf.cpp`. Eager: build a vector, then loop. A view pipeline: `split`, `filter`, `transform`, no allocation, fully inlinable. A generator: one frame allocation, then a suspend and resume per element.

ASK: "In a Release build, which one wins?" Expect "the view." Say: "Yes. And the generator's cost is roughly an indirect call per element, plus one allocation for the frame."

The deck's numbers: Debug build, GCC 14: eager 768 ms, view 936, generator 926. Take the ordering, not the ratios: it's a Debug build, and the eager version's allocations are amortized over millions of elements.

DO: Optional, if you built `build-rel`:

`./build-rel/demos/s05/demo_s05_generator_perf`. Expect three lines, each with `sum=999000000` and a time in microseconds. Point at the sums first: "same answer three ways." Then the times. Recovery: if the generator line says "not available," you're on `libc++`.

HALO, heap allocation elision, can remove the frame allocation when the coroutine is inlined into its consumer. Clang does it sometimes. Don't expect it from GCC. (pause) **Never rely on HALO for a hard real-time budget.** If you need control, `std::generator` takes an allocator, passed with `std::allocator_arg` as the first argument.

IF AHEAD: To verify HALO yourself, look for the call to `operator new` in the optimized assembly on Compiler Explorer: if the coroutine was elided, it's gone. A lambda-free, non-escaping generator consumed in the same function is the best case for Clang. As soon as the generator object is returned, stored, or passed through a non-inlined function, assume the allocation is there.

IF BEHIND: "Views win in Release; a generator is an allocation plus an indirect call per element; don't count on HALO."

31. Availability • C++23 • target 1:01, ~1.5 min

The language part, `co_await` and `<coroutine>`, is everywhere: GCC, Clang, MSVC. The library part is one implementation. `std::generator` is in `libstdc++ 14`. Not in `libc++ 18`. Not in MSVC 19.3x.

HALO, per the table, varies; the safe reading is "don't plan on it."

The exercise gates the generator: `__has_include(<generator>)` and `__cpp_lib_generator` together define `TELEMETRY_HAS_GENERATOR`. The coroutine parser and its test compile only there.

This is the strongest case in the course for feature-test macros: the same source builds on both toolchains, and the test suite differs by one test. (pause)

For `libc++` today, a third-party generator is a single header: `cppcoro`'s, or the reference implementation from the paper.

DO: Optional, 20 seconds: switch to the Compiler Explorer tab with `generator_basics.cpp` on GCC 14 and Clang 18 `libc++` side by side. One prints Fibonacci, the other prints "not available on this standard library." Recovery: if Compiler Explorer is slow, skip it; the table says the same thing.

IF AHEAD: The classic feature-test bug: `__cpp_lib_generator` is defined by the library header, not by the compiler. If you test it before including `<generator>` or `<version>`, it's undefined everywhere, and your gated code silently never compiles,

on any toolchain. Include `<version>` first, then test. A useful CI check is a test that fails if the gate is off on the toolchain you expect to have the feature.

32. Where coroutines pay off, and where they do not • C++20 • target 1:02, ~1.5 min

The middle column is the honest answer to "should we use coroutines."

Lazy sequences over I/O or expensive parsing: yes, `std::generator`, today, on `libstdc++`. Protocol decoders and state machines: yes, with one small type. Async I/O: yes, through a library, or wait for `std::execution`.

A hot inner loop over a vector: no. A view or a plain loop; the suspend and resume is pure overhead. Hard real-time with an allocation budget: not unless you've verified HALO in the generated code or you're using a custom allocator. Replacing every callback in an existing codebase: not yet. Pick one subsystem, measure, keep the callback interface behind it.

The lazy sequence and the state machine are the safe first uses, because they're single-threaded and the type is small. (pause)

ASK: "Where in your current code is there a hand-written state machine fed one input at a time?" Expect "message parsers," "bus protocols," "a mode manager." Say: "That's your tier 3 candidate for the worksheet later."

33. Coroutines takeaway • target 1:04, ~1 min

A coroutine is a control-flow tool: a function that can pause. `std::generator` is the standard's first use of it, and it's a range, so everything from Session 4 applies.

The machinery is one promise type and three awaiter methods. (beat) Use a library's, not your own, except to learn. (pause)

The last big feature is the one that changes the build.

34. What modules fix • C++20 • target 1:05, ~2 min

Every translation unit re-parses `<vector>`. Every macro in every header leaks into every file included after it.

Five rows. Parse cost: a header is re-parsed by every TU that includes it, transitively; a module interface is compiled once to a BMI, a built module interface, and imported. Macro leakage: a `#define min` in one header breaks the next one; **macros don't cross import**. Include order: imports are order-independent. Private helpers: instead of namespace detail and hope, **not exported means not visible**. ODR: one definition, one owner.

What they don't fix: they're not packages, not a build system, not faster link times, and not a reason to rewrite a working library.

ASK: "Who has actually measured where your build time goes?" Expect few hands. (pause) Say: "Hold that thought. It's step one of the adoption posture in a few slides."

Honest framing: as of 2026, modules are the least-adopted feature in this course, and the reasons are tooling, not the language. The next few slides show you why.

The macro row is the one that bites long-lived codebases. A vendor header that defines `min` and `max`, or `ERROR`, or `TRUE`, breaks code that has nothing to do with it, depending only on include order. You've all seen a file that only compiles if one header comes before another, with a comment saying "do not reorder." Modules make that whole class of problem impossible across an `import` boundary.

IF AHEAD: A related point: modules aren't the only fix for parse cost. Precompiled headers already exist in every compiler and CMake supports them with `target_precompile_headers`. For an existing codebase, PCH is often most of the compile-time win with none of the build-system risk. Modules win on hygiene, macros and visibility, which PCH doesn't touch.

35. `export module`, `export`, `import` • C++20 • target 1:07, ~3 min

Left: the CRC from Session 3 as a header. `#pragma once`, three includes, a `detail` namespace with the polynomial and the table builder, the comment "please do not use," and `crc16`. Every `#include` of this re-parses `<array>` and `<string_view>`.

Right: `demos/s05/modules/crc.cppm`, abridged. Three keywords to notice.

Line 1: `module`; on its own. That opens the global module fragment. The includes go there. Next slide is about why.

Then `export module telemetry.crc`; **This file is the module**. From this line on, nothing leaks unless it's exported.

Inside the namespace: `kPolynomial` and `kTable` have no `export`. They're module-private. An importer can't name them. No `detail` namespace needed; the language enforces what the comment used to ask for. (pause) Then `export constexpr std::uint16_t crc16`: that's the interface.

Bottom right, `main.cpp`. `import telemetry.crc`; No header, no include guard, no macros. And the `static_assert` that `crc16("123456789") == 0x29B1`, which is the standard check value for this CRC variant. **constexpr crosses the module boundary: the importer can evaluate it at compile time.**

DO: Point, don't run yet: put `modules/crc.cppm` on the right and scroll from `module`; to `export module` to the `export` on `crc16`. Then show `modules/main.cpp`. We build it on slide 39.

A naming note: the dot in `telemetry.crc` means nothing to the language. It's a convention. `telemetry.crc` isn't a child of `telemetry`; it's just a name with a dot in it.

IF AHEAD: "Not exported" means not visible, but not necessarily unreachable. If an exported function returns a type that wasn't exported, importers can still use the object through `auto`, they just can't name the type. That's sometimes useful and sometimes surprising. For a reviewer, the question for a module interface is the same as for a header's public API: is every exported name something we're willing to support forever?

IF BEHIND: "module; for includes, export module names the unit, export marks the API; everything else is private."

36. The global module fragment, and a GCC 14 rule • C++20 • target 1:10, ~2 min

The standard library is still headers until `import std;`. So where do the includes go in a module?

Look at the first snippet, three numbered parts. One: `module;`, then only `#includes`. Those declarations attach to the global module, the same as in any ordinary file, not to `telemetry.crc`. Two: `export module telemetry.crc;` starts the module purview. **From here, everything belongs to the module.** Three: imports of other modules come first inside the purview, before any other declaration.

Why not put the `#includes` after `export module`? Because then `std::array` would be declared as belonging to your module, and it would clash with every other TU's `std::array`. (pause)

Second snippet, the consumer side. On GCC 14, **#include must come before import in a consumer**. The `#include <print>` above `import telemetry.crc;`. Put the include after the import and GCC 14 errors out, because it sees the header possibly re-declaring things the module's global fragment already attached. Clang 18 accepts either order.

The rule that keeps both compilers happy: includes first, everywhere.

That's exercise task 6: swap the two lines in `main.cpp` and read GCC's error. It's worth seeing once so you recognize it.

IF BEHIND: "Includes go in the global module fragment above `export module`; in consumers, includes before imports for GCC 14."

37. Partitions, implementation units, header units • C++20 • target 1:12, ~1.5 min

One interface file per library doesn't scale. Three ways to split.

A partition: `export module telemetry:crc;`. Note the colon. Part of module `telemetry`, compiled separately, invisible outside the module except through its parent.

The primary interface, `export module telemetry;`, re-exports its partitions with `export import :crc;` and `export import :parse;`. Consumers just `import telemetry;`.

An implementation unit: `module telemetry;` with no `export`. It's like a `.cpp` file: it sees everything in the module, and the definitions live there.

And the bottom snippet: `import <vector>;` is a header unit. **A header compiled as if it were a module, and macros do cross.** That's the migration bridge for third-party headers. It's also the least supported piece: CMake has no official header-unit support as of 3.30. (pause)

IF BEHIND: Fold into slide 36: "Partitions split the interface, implementation units split definitions, header units bridge old headers but are poorly supported."

38. `import std`; • C++23 • target 1:13, ~2 min

The whole standard library as one module, and the first modules feature most codebases will switch on.

`import std`; gives you everything in namespace `std`. `import std.compat`; adds the C library's global names, `::printf`, `::size_t`, for code that uses them unqualified.

What it needs, per the table: `libc++` 17 or later, `MSVC` 17.5 or later, `libstdc++` 15. Not 14. CMake 3.30 with the experimental `CMAKE_EXPERIMENTAL_CXX_IMPORT_STD` flag and `CXX_MODULE_STD ON`. And Ninja or Visual Studio as the generator.

Measured: Microsoft reports a hello-world about ten times faster to compile, and a real TU typically 20 to 50 percent. Those are their numbers on their setup; measure yours.

The pragmatic entry point: no code changes beyond replacing a block of includes, and the compile-time win is immediate. (pause)

The blocker for anyone on GCC 14 is `libstdc++` 14; GCC 15 adds it. Clang 18 with `libc++` can do it today with the experimental CMake flag. The repo doesn't enable it because its CMake floor is 3.28.

ASK: "What doesn't come through `import std`;" Expect silence or "macros." Say: "Macros. `assert`, `INT_MAX`, `errno`, `offsetof`. You still `#include <cassert>` and `<climits>` for those."

IF AHEAD: Because `import std`; exports no macros, the feature-test macros don't come with it either. You still `#include <version>` to test `__cpp_lib_generator`. And the order rule from slide 36 applies: on toolchains with GCC-like behavior, that include goes before the import.

39. Build systems • C++20 • target 1:15, ~4 min

A module must be compiled before anything that imports it, and the build system has to discover that order by reading the source. That's the whole problem in one sentence.

The good news is in the CMake snippet: two lines. `add_executable`, then `target_sources` with `FILE_SET CXX_MODULES` listing `crc.cppm`. CMake 3.28 or later, and Ninja.

How it works: P1689. The compiler scans each source and emits what it exports and what it imports. CMake builds the dependency graph from that before compiling. That's why Ninja, or MSBuild, is required: **Make can't express dependencies discovered during the build.** (pause) If your program builds with Makefiles, that's your blocker, before anything about compilers.

Other build systems: Meson and Bazel partial, `build2` and `xmake` full.

And BMIs are compiler-specific and flag-specific. No sharing a BMI across compilers or across different `-std` flags.

DO: In the terminal: `cmake --build build-mod` (configured before class with `-G Ninja -DCOURSE_MODULES=ON`). Expect a Ninja build line that includes a scan step

before compiling `crc.cppm`. Then run the binary:
`./build-mod/demos/s05/modules/demo_s05_modules` (if the path differs, find `build-mod -name demo_s05_modules`). Expect 4F54, the CRC of "telemetry" in hex. Point out the `static_assert` passed at compile time or there'd be no binary.

DO: Then show the scan output: `find build-mod -name '*.ddi' | head -3` and open one in the editor. Point at the `provides` entry with `telemetry.crc` for `crc.cppm`, and the `requires` entry for `main.cpp`. Say: "That's P1689: the compiler telling the build system the order." Recovery: if no `.ddi` files show up, say "CMake keeps the scan results under `CMakeFiles`; the point is that a scan step ran before compile," and move on.

DO: Recovery for the build itself: if `cmake --build build-mod` fails, run `cmake -S . -B build-mod -G Ninja -DCOURSE_MODULES=ON` again and check the output says Ninja, not Unix Makefiles. If it still fails, don't debug live; say "the matrix row says it builds on GCC 14 and Clang 18; I'll post the log," and continue.

IF AHEAD: Two practical details. CMake needs Ninja 1.11 or newer for the dynamic dependency feature modules use. And on GCC 14 modules require `-fmodules-ts`; CMake adds it for module targets automatically, but a hand-written Makefile or a vendor build script won't, which is how people conclude "GCC doesn't support modules" when it does.

IF BEHIND: Build and run only, skip the `.ddi` files. "The compiler scans, CMake orders, Ninja builds. Make can't."

40. Compiler status, and what the repo does • target 1:19, ~1.5 min

This is the support matrix row for modules, expanded.

Named modules: GCC 14 yes, with the includes-first rule. Clang 18 yes, with the best diagnostics. MSVC yes, the most complete. Partitions and implementation units: all three. Header units: partial on GCC, yes on Clang and MSVC. `import std;` GCC 15, libcpp 17 with CMake 3.30, MSVC 17.5. CMake file sets: 3.28 everywhere.

What the repo does: `demos/s05/modules/` builds `telemetry.crc` on both GCC 14 and Clang 18, with Ninja and `-DCOURSE_MODULES=ON`. It's off by default so the default build keeps working without Ninja.

The compilers are ready. The reason to hesitate is your build system and your dependencies' headers. (pause)

IF BEHIND: "Compilers are ready; the build and third-party headers are the blockers."

41. An adoption posture • C++20 • target 1:21, ~3 min

Six steps, and the order matters.

One: measure the build first. `-ftime-report` on GCC, `-ftime-trace` on Clang with `ClangBuildAnalyzer` to summarize it. If headers aren't the cost, modules won't be the win. A lot

of large builds are dominated by template instantiation or by link time, and modules help neither much.

Two: `import std`; as soon as the toolchain floor allows. Zero source change beyond the includes. The biggest single win.

Three: new leaf libraries as modules, behind a build option, with a header fallback until every consumer can import. That's exactly what the repo does with `COURSE_MODULES`.

Four: header units for third-party headers where the build supports them. Otherwise leave them in the global module fragment.

Five: don't convert a working header-only library for its own sake.

Six: one compiler at a time. BMIs aren't portable, so a CI that builds with two compilers builds the modules twice.

Notice what's missing from this list: "convert the codebase." Header files aren't going away. Every module you write will sit next to headers for years, and the global module fragment exists precisely so the two can live together. The realistic end state for a defense program over the next five years is `import std`;, a few new libraries as modules, and everything else as headers. That's fine. That's a win.

Nobody here should plan a module conversion of an existing library this year unless the build measurement says headers dominate. (pause)

ASK: "For a program on a certified toolchain that gets upgraded every few years, which of these six can you do now?" Expect "one, measure" and maybe "three, behind an option." Say: "Right. Measuring is free, and it tells you whether the other five are worth arguing for at the next toolchain upgrade."

IF AHEAD: A concrete measurement recipe: build once clean with `-ftime-trace`, feed the directory to `ClangBuildAnalyzer`, and read its "expensive headers" list. If the top ten headers account for a third or more of front-end time, `import std`; and PCH will show up in your wall clock. If the top items are template instantiations, look at Session 3's concepts and explicit instantiation instead.

42. Modules takeaway • target 1:24, ~1 min

Understand modules now: `module`;, `export module`, `export`, `import`, and the build order the compiler must discover.

Adopt them where the build is ready, and expect `import std`; to be the first thing worth switching on. (pause)

Five minutes now on what was removed while all this was being added.

43. Deprecated and removed, C++14 through C++23 • target 1:25, ~1.5 min

Don't read the table; it's a reference. The pattern is what matters.

C++17 removed `auto_ptr`, `random_shuffle`, `bind1st` and `bind2nd`, `ptr_fun`, `unary_function`. Use `unique_ptr`, `shuffle`, lambdas. Dynamic exception specifications are gone: `noexcept`. C++20 removed `result_of` and `uncaught_exception`. `<codecvt>` and `stringstream` are removed in C++26.

One row matters to embedded people more than the rest: `volatile`. (beat) C++20 deprecated compound operations on `volatile`. C++23 restored part of it, and the part matters. **C++23 undeprecated the bitwise compound assignments, `|=`, `&=`, `^=`, the register-twiddling ones. `++` and `--` on a `volatile` are still deprecated, and both GCC and Clang still warn in C++23 mode.** (pause) So `REG |= MASK` is fine again. `++counter` on a `volatile` is not, and `+=` isn't either by the standard's wording, even where the compiler stays quiet.

And `volatile` was never for threads. If you see a `volatile` counter shared between threads, it wants to be an `atomic`.

The other notable one: `libstdc++` keeps most removed names available with a deprecation warning. `libc++` removes them. So a codebase that only builds on GCC may be carrying `auto_ptr` and not know it.

Let's find them.

IF BEHIND: "Removed names: `libstdc++` warns, `libc++` refuses. And C++23 restored `volatile` `|=`, `&=`, `^=`, not `++`."

44. Finding them • demo • target 1:26, ~3 min

The snippet is `demos/s05/deprecated.cpp`, the modern function: each line is the replacement, with the old spelling in the comment. `make_unique` for `auto_ptr`. `shuffle` with a seeded `mt19937` for `random_shuffle`. A lambda for `bind2nd`. `invoke_result_t` for `result_of`. Five member typedefs instead of inheriting `std::iterator`. `alignas` plus a `std::byte` array for `aligned_storage`.

The interesting part is under `SHOW_ERRORS`: the old spellings.

DO: Run the two compiler lines from the slide, one after the other: `g++-14 -std=c++23 -DSHOW_ERRORS -Wdeprecated -fsyntax-only demos/s05/deprecated.cpp`, then `clang++ -std=c++23 -stdlib=libc++ -DSHOW_ERRORS -fsyntax-only demos/s05/deprecated.cpp`. Expect deprecation warnings from GCC for `auto_ptr`, `random_shuffle`, `bind2nd`, `result_of`, `std::iterator`, `aligned_storage`, and the `volatile ++`. Expect hard errors from Clang with `libc++`: no member named '`auto_ptr`' in namespace '`std`' and friends. Point at the same line in both outputs: "warning on one library, error on the other." Recovery: if a command fails for an unrelated reason, switch to the preloaded Compiler Explorer tab with the same code on both compilers.

ASK: "Which result would you rather have in CI?" Expect "the error." Say: **"Removed names are a toolchain test: build on both."**

DO: Then the metric: `clang-tidy -p build --checks='modernize-*' exercises/s01-*/starter/src/*.cpp | grep -c warning`, and the same on the Session 5 solution's sources. Expect a large number for the starter and near

zero for the solution. Say: "That difference is the course." Recovery: if clang-tidy says it can't find a compilation database, read the two numbers off your sticky note.

Exercise task 7 is exactly that comparison.

IF AHEAD: The library deprecations come through `-Wdeprecated-declarations`, which is on by default on both compilers; `-Wdeprecated` is a separate group for deprecated language features. Once the count is zero, `-Werror=deprecated-declarations` turns a regression into a build break. And `--checks='modernize-*'` adds to whatever `.clang-tidy` already enables; for a pure modernize count use `--checks='-*,modernize-*'`.

IF BEHIND: Run only the Clang line, show one error, and give the clang-tidy numbers from the sticky note.

45. Deprecations takeaway • target 1:29, ~0.5 min

Removed names are a toolchain test: libstdc++ warns, libc++ refuses. Build on both.

`-Werror=deprecated-declarations` once the count is zero, and clang-tidy `modernize-*` as the metric for how far you've come. (pause)

Now the workshop. This is the part that matters most.

46. Three tiers • C++23 • target 1:30, ~2 min

Twenty minutes, and these twenty are about your code.

Three tiers. **Tier 1, mechanical and zero risk, this month.** Local edits, no interface changes, clang-tidy does most of it: `make_unique`, `[[nodiscard]]`, structured bindings, `string_view` at read-only boundaries, optional for sentinels, using over `typedef`, the spaceship, `scoped_lock` for `lock_guard`, `jthread` for `thread`. On that last one, remember slide 10: mechanical, but check each thread's exit path, because the destructor now joins.

Tier 2, interface changes, this quarter. Callers change, so the tests have to exist first. span for buffers, expected for errors, concepts on public templates, format and print, `constexpr` tables, projections.

Tier 3, architectural: evaluate, and pilot one. View pipelines in hot paths, `stop_token` cancellation, a semaphore queue replacing a hand-written one, `std::generator` for a lazy source, modules for one new library.

Tier 1 is Session 1's exercise. Tier 2 is Sessions 2 through 4. Tier 3 is today. (pause)

Tier 1 needs no meeting. Tier 2 needs tests. Tier 3 needs a pilot and a measurement.

Why tiers at all? Because the most common way a modernization effort dies is a big-bang branch. Somebody spends three months converting everything, the branch drifts from mainline, the merge is terrifying, and the effort gets cancelled. Tiers keep every change small enough to merge the same week. The `report_identical` test in this course is the model: every step lands, every step is behavior-preserving, and the test says so.

ASK: "What's the tier 2 item you'd get the most pushback on in your program's review board?" Expect "expected" or "changing APIs." Say: "Usually it's anything that changes an interface another team consumes. That's why tier 2 is a quarter, not a month."

IF BEHIND: "Tier 1 this month, no meeting. Tier 2 this quarter, tests first. Tier 3 one pilot with a measurement."

47. Tooling as a force multiplier • C++23 • target 1:32, ~1.5 min

Six tools. clang-tidy modernize-*, performance-*, readability-* with -fix. Run it per directory, commit per check, so each commit is one kind of change and reviewable.

The one to underline: the flag progression. A CI job on the next standard, -std=c++20 while the default is still 17, allowed to fail. That's how you find out what the switch will cost before you commit to it. (pause) On a program with a certified toolchain, it's also how you build the evidence for the next upgrade.

Warnings as errors per directory: the course's course_warnings target, applied to new code first, then outward. Feature-test macros for the mixed-toolchain years; the exercise uses them in three places. Sanitizers in CI: ASan and UBSan on every test, TSan on every concurrent test, as separate jobs. And the support matrix as a living document: add a row when you gate a feature, delete it when the floor rises.

IF BEHIND: "clang-tidy per directory, a CI job one standard ahead, sanitizers as separate jobs."

48. Writing the coding-standard update • C++23 • target 1:33, ~1.5 min

One page. Three verbs.

Mandate for tier 1: "New code uses make_unique; new outside a constructor is a review comment." "Value-returning functions are [[nodiscard]]."

Allow for tier 2: "std::expected is the error type for new interfaces; existing bool plus out-parameter APIs are converted when touched."

Pilot for tier 3: "Ranges pipelines in telemetry/report through Q4; a decision by the retrospective."

Reference the C++ Core Guidelines by rule number instead of restating them: R.11, avoid calling new and delete explicitly; F.20, prefer return values to output parameters; ES.20, always initialize an object. Today added CP.52 and CP.53 for coroutines.

Reject anything that needs more than one page: it won't be read. (pause) Everyone in this room has seen a thirty-page coding standard. A one-page document with three verbs gets applied in review. Anything longer gets linked and ignored. If your program is bound to MISRA or AUTOSAR, this page sits on top of that, as project rules and documented deviations, not instead of it.

Now you write one.

49. The worksheet • demo • target 1:35, ~13.5 min

DO: Paste `handouts/adoption-roadmap-template.md` into the Teams chat, or post the path. Start a visible 8-minute timer.

Open the worksheet. Eight minutes. Three things.

One: name a codebase you own and its current `-std=` flag. Your real one.

Two: fill in the tier 1 table. For each row: where, a directory or a module; who; and whether clang-tidy can do it. The template's tier 1 rows match the slide, using, the spaceship, `scoped_lock` and `jthread` included.

Three: pick one tier 2 row and one tier 3 row, and write the pilot area.

Then three volunteers read theirs, and the rest of us ask one question each: **what would break?** (pause)

Go.

DO: See "Exercise coaching" below for what to watch for and say while they write. At 5 minutes left, make the call. At 0, take three volunteers, about a minute each plus one "what would break?" question.

Close it out by saying: "The 'what would break' question is the review your roadmap needs before it becomes a ticket. Ask it again on Monday, with your lead."

50. The capstone • C++23 • target 1:48, ~1.5 min

The worksheet is the plan. The capstone is the proof.

`exercises/capstone/README.md`. Pick 300 to 500 lines of code you own, with tests, or write the tests first. Apply tier 1, then tier 2, one commit per feature. Deliver a before-and-after diff and one paragraph: what changed, what the tests said, and what you wouldn't do again.

Two weeks. Bring the paragraph to the retrospective.

The course's five exercises are the worked example: the same program, five diffs, one unchanged report. (pause)

Small enough to finish, real enough to matter. And I'll review your diffs: send them over.

IF BEHIND: "300 to 500 lines, tier 1 then tier 2, one commit per feature, a diff and a paragraph in two weeks. I'll review them."

51. What is coming in C++26 • C++23 • target 1:50, ~4 min

Last ten minutes. What's next, and then the whole course in one slide.

C++26 is feature-complete; the technical work finished in 2025, and the first compiler support is landing now. Check the `cppreference` table for what your compiler has.

Reflection, P2996: the biggest change since templates. The `constexpr` operator gives you a compile-time handle to a type, splicers written `[ : ]` turn it back into code, and `std::meta` lets you enumerate members. The first uses everyone writes are serialization and enum-to-string. For a telemetry program, think: a record type that serializes itself without a hand-maintained field list. Watch it.

Contracts: pre, post, and `contract_assert`, with enforcement modes chosen at build time, ignore, observe, or enforce. The replacement for `assert`, and for half your comments that say "caller guarantees." For code that has to argue its correctness to a reviewer or an assessor, preconditions in the signature are a big deal. Watch it.

std::execution, P2300: senders, receivers, and schedulers. That's the one that changes today's concurrency story: the thread pool and the async model we said were missing. Wait for the `stdexec` library implementation to settle.

The hardened standard library: bounds-checked `operator[]` and friends as a build mode. **That's the free win: turn it on the day your library ships it.** (pause)

Why is it free? Because most bounds bugs are in code nobody is going to rewrite. You don't change a line; you change a build flag, and an out-of-range index becomes a defined trap instead of silent memory corruption. Several standard libraries already have a version of this as a vendor mode today; C++26 standardizes it. If your program has a test build, turn it on there first.

`inplace_vector`, a fixed-capacity vector with no heap, and `hive`, a container with stable addresses. Embedded and game codebases have wanted both for years. `#embed` for binary resources, `constexpr` exceptions, and `std::simd`: use when available.

ASK: "Which one of these would change your program the most?" Expect "contracts," "inplace_vector," or "hardened library." Say: "Good. Any of those is a reason to keep the CI job one standard ahead running."

IF AHEAD: `inplace_vector` deserves a sentence for this audience. Its capacity is a template parameter, the storage is inside the object, and it never allocates. If your coding standard forbids heap after initialization, it's the first standard container you can use freely, and you can already get the same semantics from Boost's `static_vector` today, which makes it an easy tier 2 candidate to pilot ahead of the standard.

52. Where to follow the language • C++23 • target 1:54, ~2 min

Six places.

The cppreference compiler support table: the first page to open when a feature doesn't compile. It answers most of the questions this course got.

isocpp.org for the trip reports after each WG21 meeting, three a year: what passed, and what didn't. The papers themselves: `wg21.link` slash the paper number takes you to any paper on today's slides: P2300 for execution, P2996 for reflection, P2502 for generator. They're more

readable than you'd expect; the abstract and the motivation section are written for practitioners. (pause)

Talks: the CppCon and ACCU list in the syllabus, one per session topic. Compiler Explorer for "does this compile on X," and the CI job on the next standard for "does our code."

And this repo: the feature timeline, the support matrix, the cheat sheets, and five exercises with solutions. It stays available. Use it as a reference, not a souvenir.

53. Course takeaway · target 1:56, ~4 min

Five sessions ago we started with a careful, idiomatic C++11 program. Not bad code. Good C++11.

The program is unchanged. Five sessions, five diffs, one report, byte for byte. (pause)

And everything about how it's written is different. Five phrases, one per session.

Ownership in the type. Session 1: `unique_ptr` instead of `new`, and the type tells you who frees it.

Errors in the return. Session 2: `expected` instead of a `bool` and an out-parameter, so an error can't be ignored by accident.

Work at compile time. Session 3: the CRC table built by the compiler, inputs checked by `constexpr`, templates constrained by concepts.

Algorithms without iterator pairs. Session 4: projections, views, pipelines.

Threads that join and cancel themselves. Today: `jthread`, `stop_token`, a queue on semaphores, and a parser that's a generator. (pause)

Notice the shape of that list. Every item moves a rule out of a comment, or out of someone's head, and into something the compiler or a tool enforces. "Caller must free this" became a type. "Check the return value" became `[[nodiscard]]` and `expected`. "Table must match the polynomial" became a `static_assert`. "Remember to join" became a destructor. And "I think this is race-free" became a `TSan` job in CI. (pause)

None of that changed what the program does. All of it changed what a reviewer has to check, and what the compiler checks for you. That's the real argument for modern C++ in a long-lived codebase: fewer things a person has to get right by hand.

So, Monday morning. Three things. **One tier 1 item. One clang-tidy run. One CI job on the next standard.** (pause) That's it. Not a migration plan. Three small things you can finish by lunch.

ASK: "Before you go: what's the one tier 1 item you're doing Monday?" Take two or three answers in chat or out loud. Say "good" and nothing more to each; let them own it.

The capstone is two weeks. Bring the paragraph to the retrospective. Send me your diffs and I'll review them.

Thank you. It's been a good five sessions. (beat) Questions for as long as you want to stay.

Exercise coaching

This session has two exercise pieces, and only one of them has a slot in the deck.

The take-home tasks (1 to 7; the README now says so)

The agenda has no block for tasks 1 to 3, and the README now lists them as the first at-home work, with the roadmap worksheet (task 8) as the in-class piece. Launch them as the first thing to do after class.

Launch in 60 seconds, at slide 21: "Tonight: `exercises/s05-concurrency/README.md`. Task 1 is the bounded queue on two semaphores, task 2 is the `jthread` producer with the calling thread as consumer, task 3 is running it under TSan. The report has to stay byte-identical. Tasks 4 to 7 are cancellation, the generator, modules, and clang-tidy. The solution is clean under TSan on both compilers; check yours against it."

Stumbles to warn about now (they will ask you later):

- `counting_semaphore`'s count is `std::ptrdiff_t`; a `size_t` capacity trips `-Wsign-conversion` under `-Werror`.
- `items_` must allow `Capacity + 1` for the extra release in `close()`; otherwise it's UB on a full queue.
- Reading `lines_read` before `producer.join()` is the race TSan is there to catch. The fix is the join, not making it atomic.
- Records must come out in input order: one producer, one FIFO queue, so they will, unless someone parallelizes parsing.
- For task 4: the `stop_callback` must close the queue as well as request the producer's stop, or a consumer blocked in `pop` sleeps through it (which is also why `pop` polls with `try_acquire_for`). Declare the callback after the producer `jthread` so it's destroyed first.
- For task 5: include `<version>` or `<generator>` before testing `__cpp_lib_generator`. On `libc++` the test simply doesn't compile in; that's expected, not a failure. Expect "next unread line is 4" to surprise people.
- For task 6: `-G Ninja` is required; Makefiles won't build modules.

Debrief: at the retrospective, or by message. Show the solution's `queue.h` (slide 12 is an excerpt), then the TSan-clean CTest run.

The roadmap worksheet (slide 49, in class)

Launch (60 seconds): paste the template, say the three steps on the slide, start a visible 10-minute timer, say "Go," and stop talking.

What to say while they work: very little. Every two or three minutes, one sentence to the room:

- "Use a real codebase and its real `-std=` flag, not an imaginary one."
- "For each tier 1 row, a directory and a name. 'Everywhere' and 'the team' aren't answers."
- "If clang-tidy can do it, say which check: `modernize-make-unique`, `modernize-use-nodiscard`, `modernize-use-using`."

What to watch for:

- Anyone with more than one tier 3 item: push back. "Pick the one you'd pilot first; the other one waits for the retrospective."
- Anyone with zero tier 1 items: push back. There's always a new somewhere.
- Tier 2 items with no tests under them: ask "what tells you the callers still work?"
- People on a C++14 or C++17 toolchain who think the worksheet doesn't apply: most of tier 1 is C++14 and C++17. `make_unique`, `[[nodiscard]]`, structured bindings, `string_view`, `optional`, `scoped_lock` all work on a C++17 floor.
- Silence in Teams: ask one person by name to unmute and say their codebase and flag; it unblocks the room.

5-minutes-left call: "Five minutes. If tier 1 has a row with a where and a who, you're on track. Now pick your one tier 2 and one tier 3."

Debrief (about 3.5 minutes): three volunteers, about a minute each. After each, one question from the room: "What would break?" If nobody asks, ask it yourself. Good answers name a test, an interface, or a team. Weak answers say "nothing"; push gently: "What does the CI job one standard ahead say?" There's no solution to show: the debrief is the question.

Closing: "Take the worksheet to your lead this week. The 'what would break' answers are your risk list. The capstone is two weeks; that's where this worksheet turns into a diff."

Likely questions and answers

1. Our certified toolchain is GCC 11 or 12 in C++17 or C++20 mode. What from today can we use? `scoped_lock` and `shared_mutex` are C++17 and available everywhere you care about. The C++20 thread primitives (`jthread`, `stop_token`, `semaphores`, `latch`, `barrier`, `atomic wait`) arrived in `libstdc++` over GCC 10 and 11, `atomic<shared_ptr>` and `move_only_function` in GCC 12, and `std::generator` only in GCC 14. Check the cplusplus library support table against your exact version before you plan; the course's support matrix covers GCC 14 and Clang 18 only.

2. Is `jthread` a drop-in replacement for `std::thread`? Almost. The difference is the destructor: `jthread` requests a stop and joins instead of calling `terminate`. A thread that never checks its token and blocks forever will now hang the destructor instead of crashing, and code that relied on `detach()` needs a look. It's tier 1 in effort but worth a reviewer checking each thread's exit path.

3. Does MISRA C++ or AUTOSAR say anything about this material? MISRA C++:2023 was written against C++17, so C++20 features like coroutines, `jthread`, `semaphores`, and modules

are outside what it was written to cover. In practice that means project rules and documented deviations, reviewed like any other. AUTOSAR's C++14 guidelines have been folded into MISRA C++:2023, so don't expect separate guidance there.

4. Can we use coroutines with no heap, or under hard real-time constraints? Yes, with work. A `promise_type` can define its own `operator new` and `operator delete` to allocate frames from a static pool, and `std::generator` takes an allocator. If the promise also defines `get_return_object_on_allocation_failure`, the frame is allocated with a non-throwing `new`. Don't rely on HALO; verify the generated code.

5. Why not just use `std::async` for background work? The future from `std::async` blocks in its destructor, so discarding it makes the call synchronous. There's no pool, no priority, and no cancellation. It's fine for a one-off computation in a test or a tool; for a system, use `pthread` with a queue, or a real pool behind an interface.

6. Can we run ThreadSanitizer on our embedded target? No. TSan needs a 64-bit host OS (Linux, macOS, FreeBSD) and lots of memory. Build the threading logic in host unit tests with standard primitives and run TSan there. It also can't be combined with ASan in one binary, so it's a separate CI job.

7. Is `std::atomic<std::shared_ptr>` lock-free? Usually not; `is_lock_free()` will typically return false, and `libstdc++` uses a small internal lock. What it gives you is that readers never wait on the writer's work, only on the pointer swap. If you need truly lock-free reads, look at C++26's RCU and hazard pointers or a library implementation.

8. Can we ship a library as a compiled module, the way we ship headers and a .a? Not the BMI. BMIs are specific to the compiler, its version, and the flags, so consumers must build them from your .cppm source. You ship the interface source plus the compiled object library, and the consumer's build system compiles the BMI.

9. Will modules speed up our build? Possibly, if header parsing dominates; measure with `-ftime-trace` and `ClangBuildAnalyzer` first. Modules don't speed up linking or template instantiation much. Precompiled headers are often most of the win today with far less build-system risk, and `import std;` is the cheapest module win once your toolchain supports it.

10. We still use `volatile` for memory-mapped registers. Is that deprecated now? No. `volatile` itself is fine and still the tool for MMIO. C++20 deprecated some compound operations on it; C++23 restored `|=`, `&=`, and `^=`. `++`, `--` (and by the standard's wording other compound operators like `+=`) on volatile objects remain deprecated; write `reg = reg + 1` to make the read and the write explicit. `volatile` is not for inter-thread communication; that's what atomics are for.

11. Does the stop token use memory ordering we need to think about? A `request_stop()` that returns true synchronizes with a `stop_requested()` call that observes it, so data written before the request is visible to the thread that sees the stop. You don't add fences. Data shared during normal operation still needs its own synchronization, like the queue's mutex and semaphores.

12. Are C++20 semaphores and atomic wait efficient on non-Linux systems? It's implementation-defined. On Linux, `libstdc++` uses `futexes`. On other platforms the

implementations fall back to other mechanisms, typically a mutex and condition variable behind a table of waiters, so the semantics hold but the cost differs. If you're on an RTOS with its own C++ library port, check what it implements before relying on it.

Deck issues found

Status: fixed in the deck, demos, outline and README on 2026-10-02 (see scripts/README.md). Items kept for the record. Still open: the Compiler Explorer <add short link> placeholders in every demo file header.

- Slide 2 / timing: the agenda sums to 120 minutes with no block for the README's "in class (about 20 minutes)" tasks 1 to 3, though slide 2's notes call them "the in-class part." Either cut those words from the README and notes or find 20 minutes.
- Slide 2 vs syllabus: the syllabus has deprecations 1:25 to 1:35, workshop 1:35, close 1:55; the deck has 1:25, 1:30, and 1:50.
- Slide 49 / timing: three teaching slides, a 10-minute worksheet, three volunteers with questions, and the capstone in 20 minutes leaves about 3.5 minutes for the volunteers.
- Slide 8 notes: `std::scoped_lock lock;` is caught by GCC's `-Wunused-variable` (verified on GCC 13), not by clang-tidy; clang-tidy's `bugprone-unused-raii` catches the sibling `std::scoped_lock{m};` temporary.
- Slide 9: the red line says "five times slower," but the measured 695 ms vs 171 ms is about 4x.
- Slide 13: the "Before (C++11)" code uses `CTAD(std::unique_lock lock(m_))`, which is C++17; C++11 needs `std::unique_lock<std::mutex>`.
- Slide 15 / `atomic_wait.cpp` line 30: the snippet reads `obj.counter` directly while `ref` is still alive, breaking its own rule (and `[atomics.ref.generic]`); use `ref.load()` or put `ref` in an inner scope.
- Slide 17: "Each << is atomic" is not guaranteed; the standard only says concurrent use of synchronized standard streams is not a data race, and characters may interleave (as the snippet's own comment says).
- Slides 30 and 31: the notes say Clang does HALO "sometimes" while the table says "often"; GCC is listed as "rarely," but GCC isn't known to have a coroutine heap-elision pass, so "do not expect it" is safer.
- Slide 39 and README task 6: `demos/s05/modules/CMakeLists.txt` is not in the staged copy (only `crc.cppm` and `main.cpp`); confirm it exists in the repo.
- Slide 43 / `deprecated.cpp` line 51: C++23 (P2327) restored only `volatile`, `&=`, `^=`; `++counter` is still deprecated (GCC 13 and Clang 18 both warn under `-std=c++23`), so the comment "un-deprecated C++23" is wrong. The table row should say which operations were restored.
- `deprecated.cpp` line 47: the `throw()` comment says "a hard error everywhere," but under `-std=c++23` GCC 13 accepts it silently and Clang 18 only warns (`-Wdeprecated-dynamic-exception-spec`).
- Slide 44 notes: library deprecations warn through `-Wdeprecated-declarations` (on by default); `-Wdeprecated` is a separate group and is not on by default in Clang.

- Slides 36 to 39: these are hand-typed excerpts, but unlike slides 12, 24, and 35 their notes don't say so (repo convention).
- Slide 46 vs `handouts/adoption-roadmap-template.md`: the slide's tier 1 has `using`, `<=>`, `scoped_lock`, and `jthread`, and tier 2 has `constexpr` tables and projections, but the template has no rows for them.
- Slides 10 and 46: "no downside" and "zero risk" for `jthread` overstate it; the destructor joins instead of terminating, so a thread that never checks its token will hang. Worth a caveat.
- Slide 50: `exercises/capstone/README.md` is not in the staged copy (`PLAN.md` lists it); confirm it exists.
- All `demos/s05/*.cpp`: the Compiler Explorer link in each header is still `<add short link>`.